

Novel View Synthesis with SynSin

Madiyar Bolatov, AAI-2501M

April 23, 2026

1 Introduction

The aim of this project was to apply the pretrained **SynSin** model to the task of novel view synthesis and evaluate its behavior on images that differ from the scenes used during training. According to Chapter 9 of 3D Deep Learning with Python, SynSin is presented as an end-to-end method for view synthesis, with emphasis on its architecture and hands-on testing workflow [1].

SynSin generates a new view of a scene from a single input image. In the original paper, the model predicts a latent 3D point cloud of features, renders this representation from a new viewpoint, and then refines the rendered result to produce the final image. The method is trained end-to-end without ground-truth 3D supervision [2].

The main purpose of this project was not only to run the pretrained model, but also to test how well it generalizes to out-of-domain images, such as a close-up face, a top-view object scene, and a single isolated object. The implementation used in this work loads the pretrained **RealEstate10K** checkpoint and performs inference on custom input images with manually selected viewpoint parameters.

2 Theoretical Background

Chapter 9 of the textbook explains SynSin as a state-of-the-art approach to end-to-end view synthesis and highlights two main practical areas: network architecture and model training/testing [1].

In the paper, SynSin takes an input image I and a relative pose transformation T . The system contains:

- a spatial feature network that predicts dense feature maps,
- a depth regressor that estimates a depth map,
- a neural point cloud renderer that projects features into a new view,
- and a refinement network that synthesizes the final output image.

The paper also explains that the method is trained with a combined loss:

$$L = \lambda_{\text{GAN}} L_{\text{GAN}} + \lambda_{l1} L_{l1} + \lambda_c L_c$$

where L_{GAN} improves image realism, L_{l1} enforces pixel-level similarity, and L_c is a content loss. The reported training weights are $\lambda_{\text{GAN}} = 1$, $\lambda_{l1} = 1$, and $\lambda_c = 10$.

In this project, however, this loss was not used directly in code, because the model was not trained from scratch. Instead, a pretrained checkpoint was loaded and used only for inference on new images. The notebook code confirms that the RealEstate10K checkpoint `./modelcheckpoints/realestate/zbufferpts.pth` was loaded and evaluated in test mode.

3 Environment Setup and Implementation

The project environment was set up locally by cloning the SynSin repository, downloading pretrained model checkpoints, and loading the **RealEstate10K** model in a Jupyter notebook. The notebook code changes the working directory to the **SynSin** project folder, inserts that path into `sys.path`, loads PyTorch, and restores the pretrained model from the checkpoint file.

The checkpoint used in this work was:

```
./modelcheckpoints/realestate/zbufferpts.pth
```

The implementation also defined an image preprocessing pipeline consisting of resizing to 256×256 , conversion to tensor form, and normalization. Then the notebook used a custom function `synthesize_view(...)` to run novel view synthesis on unseen images.

Unlike the original demo code that used quaternion-based rotation, the final implementation used `scipy.spatial.transform.Rotation` to construct the rotation matrix. This change affected only how the viewpoint transform was encoded numerically; it did not change the conceptual SynSin pipeline. The actual inference step was still performed by calling the pretrained model's `forward_angle(...)` function.

4 Experimental Procedure

4.1 Selected test images

Three input images were used in the experiment:

1. A close-up portrait of a human face
2. A top-view scene with a book and a clock
3. A single chess piece on a simple background



Figure 1: A single object: Bishop



Figure 2: A top-view scene: Book and clock



Figure 3: A close-up portrait of a human face

These images were chosen deliberately because they differ strongly from the scene-centric data used to train SynSin. The paper evaluates SynSin mainly on RealEstate10K, Matterport3D, and Replica, which contain indoor and outdoor scene views rather than close-up portraits or isolated studio-like objects.

4.2 Viewpoint transformation

The novel view was generated by specifying small transformation parameters:

$$\theta, \phi, t_x, t_y, t_z$$

In the notebook, these parameters were converted into a rotation matrix and translation vector, combined into a transform matrix, and passed to the model. The code first tested several small candidate transformations and then selected the most visually stable parameters for each image.

The final chosen settings were:

- bishop: $\theta = -0.08, \phi = 0.05, t_z = 0.08$
- books: $\theta = -0.08, \phi = -0.05, t_z = 0.08$
- terence: $\theta = -0.05, \phi = 0.03, t_z = 0.06$

5 Results

5.1 Chess piece on a simple background

This image produced the strongest result. The overall silhouette and geometry of the chess piece were preserved reasonably well. The generated novel view clearly showed a change in perspective, while the object remained recognizable.



Figure 4: Bishop result

However, some artifacts were still visible:

- slight blur near the image borders,
- mild contour smoothing,
- and a depth map that was structurally coarse rather than highly detailed.

The likely explanation is that the image contains a single centered object and a simple background. Although this type of image is not part of the RealEstate10K domain, the visual structure is still relatively simple, making the synthesis problem easier.

5.2 Top-view scene with a book and a clock

This example produced a moderate result. The general composition of the scene was preserved: the book, the clock, and the wooden surface remained recognizable in the synthesized image.



Figure 5: Book and clock result

At the same time, several artifacts were visible:

- slight geometric warping around the clock,
- reduced precision in the rectangular edges of the book,
- and texture blur in the background.

This result suggests that SynSin can preserve the global layout of an unfamiliar scene, but it struggles more when the viewpoint is unusual. A top-view composition is less consistent with the camera trajectories and scene structure typical of the RealEstate10K training data.

5.3 Close-up face

The face image produced the weakest result. The identity of the subject remained recognizable, but the new view contained visible distortions around the glasses, face contour, hairline, and background.

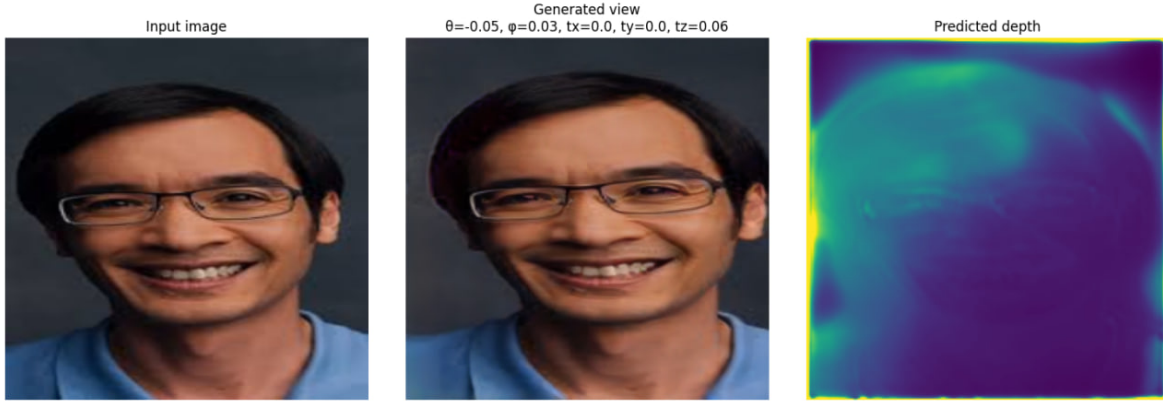


Figure 6: Close-up face result

The main artifacts were:

- facial warping,
- softening of local features,
- unstable boundary reconstruction,
- and a weak, overly smooth depth map.

This is expected because human faces require fine-grained geometric consistency. The SynSin paper focuses on indoor and outdoor scene datasets, not close-up portrait synthesis, so this image represents the strongest domain shift among the three.

6 Comparison with typical SynSin results from the paper

To compare these outputs with in-domain performance, I referred to the qualitative examples shown in the SynSin paper. Figure 7 presents qualitative results on RealEstate10K. These images show that SynSin performs much better on typical scene-based inputs such as rooms, hallways, furniture layouts, and property walkthrough views

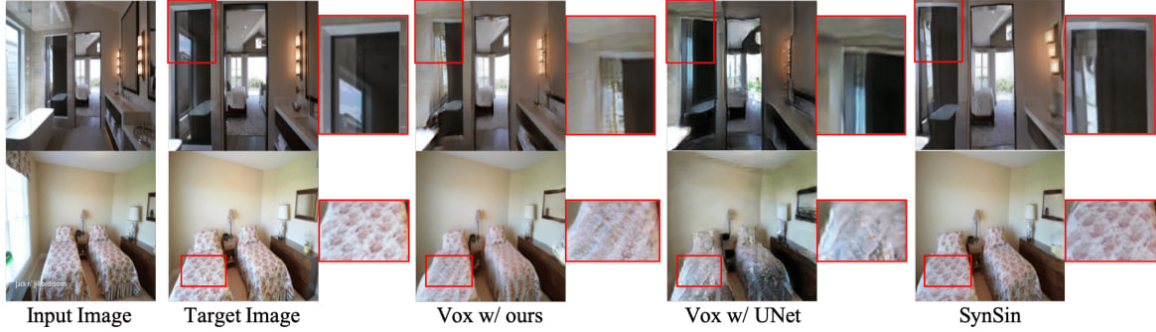


Figure 7: Qualitative results on RealEstate for ours and baseline methods. Given the input view and the camera parameters, the methods are tasked to produce the target image. The red squares denote interesting differences between the methods. In the upper row, our model better recreates the true 3D; in the bottom row, our model is better able to preserve detail.

Compared with those examples, my own results contain more visible artifacts. This difference is understandable because my three test images are not scene-centric

7 Conclusion

In this project, I successfully applied the pretrained SynSin model to novel view synthesis on three custom images that differ significantly from the model’s original training domain.

The experiment showed that:

1. SynSin can still produce plausible viewpoint changes on out-of-domain images;
2. performance is best when the image has a simple object-centered structure;
3. performance degrades for unusual viewpoints and especially for close-up human faces.

Among the tested examples, the chess piece produced the best result, the top-view book-and-clock scene produced a moderate result, and the portrait face produced the weakest result. Compared with the qualitative examples in the SynSin paper, my outputs showed more visible artifacts, which can be explained by the mismatch between my chosen images and the original training distribution.

References

- [1] X. Ma, V. Hegde, and L. Yolyan, *3D Deep Learning with Python: Design and develop your computer vision model with 3D data using PyTorch3D and more*. Packt Publishing Ltd, 2022.
- [2] O. Wiles, G. Gkioxari, R. Szeliski, and J. Johnson, “Synsin: End-to-end view synthesis from a single image,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 7467–7477, 2020.